

Introduzione ai container

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

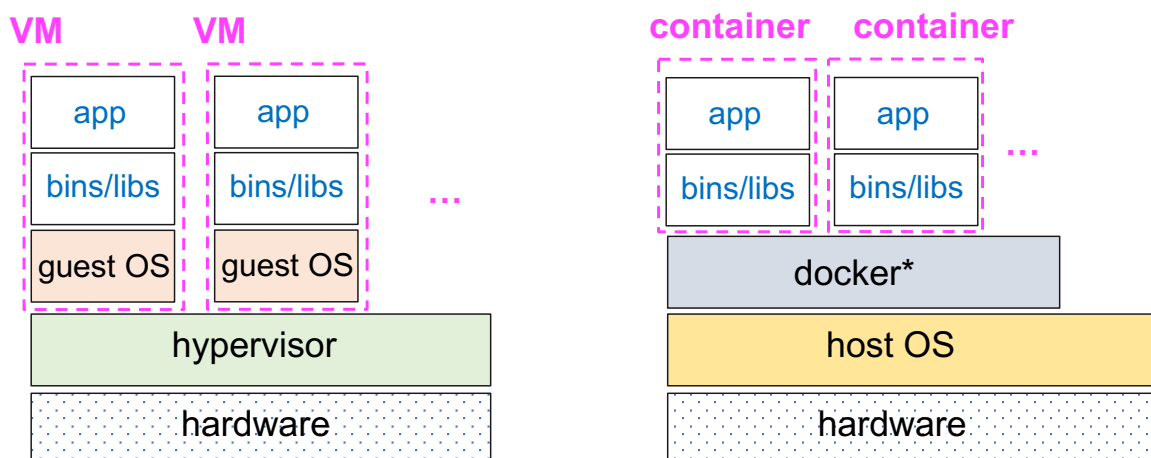
Virtual machine

- Una **virtual machine** (VM) ospita un **intero** sistema operativo (il guest OS) separato dall'host:
 - trasforma l'hardware in **hardware virtuale**;
 - isolamento delle VM;
 - esecuzione di OS differenti;
 - supporto a *migration*, *aggregation*, etc.

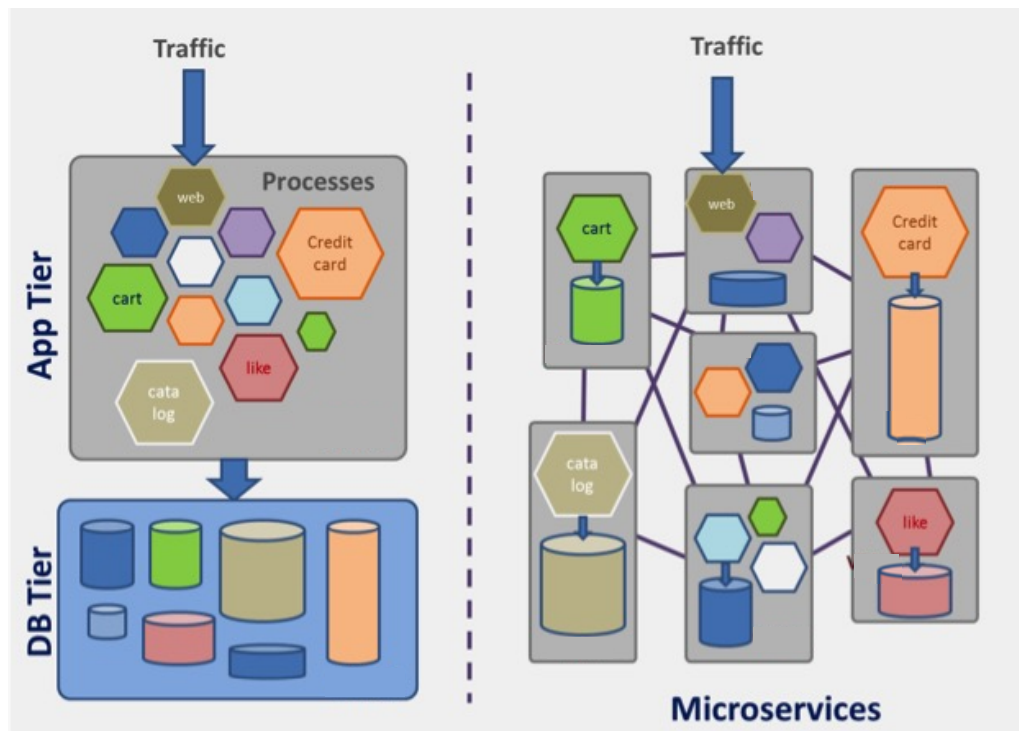
Alcune considerazioni sui container

- ❑ I container sono un modo per **isolare** processi e risorse:
 - un container incapsula una **applicazione** e tutte le sue **dipendenze** (librerie, binari, file di configurazione, etc.);
- ❑ Un container **non** virtualizza alcun HW e condivide il kernel con l'host (ciascun container condivide il SO con gli altri container);
- ❑ Un container è molto più "piccolo" di una VM;
- ❑ **Pro**: ciclo di vita, distribuzione, overhead; **contro**: configurazione, sicurezza, isolamento.

VM vs container (rappresentazione)



* Nel mondo LINUX esistono diverse tecnologie di **virtualizzazione "leggera"**; **LXC** (Linux Containers) e **docker** – oggetto della lezione – sono i prodotti più popolari.



Source: <https://www.sdgggroup.com>

Alcune considerazioni sui container

□ Meccanismi di base*:

- **chroot**: system call UNIX per cambiare la root directory di un processo e dei suoi figli a una nuova locazione del file system;
- **namespaces**: ogni container ha una istanza di ogni tipo di namespace (pid, rete, ...) per limitare gli oggetti host, visibili;
- **cgroup** (control group): permette di organizzare processi in gruppi, monitorare e limitare l'uso di vari tipi di risorse (tempo di cpu, memoria, disco, banda di rete);
- **cpuset**: consente di associare CPU (core) e sottoinsiemi di memoria a gruppi di processi.

*Descritti brevemente al [Cap 7 par. 7.11](#)

Docker

- ❑ Piattaforma open source per la creazione, gestione, e orchestrazione di **container** per Linux;
- ❑ Portabile tra macchine (che supportano docker);
- ❑ L'**obiettivo** di docker è quello di ottimizzare lo sviluppo, testing, rilascio e distribuzione di applicazioni.



Container:
Docker:

Cap 7 par. 7.11
<https://docs.docker.com/guides/get-started/>

Docker: installazione

```
sudo apt-get update
sudo apt-get install docker.io
sudo usermod -aG docker $USER
su $USER
```

```
# Opzionali, al fine di verificare installazione:
sudo systemctl status docker
```

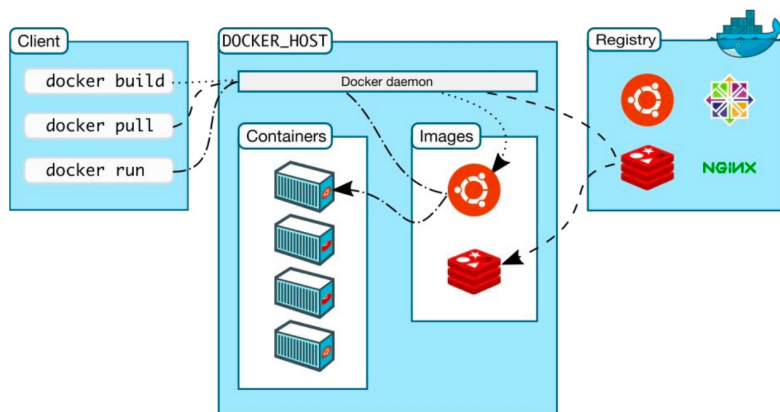
```
docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
------------	-----	----------	---------	------

```
docker run hello-world
```

Docker

- ❑ Architettura **client-server** per la gestione di immagini, containers, volumi e reti virtuali.

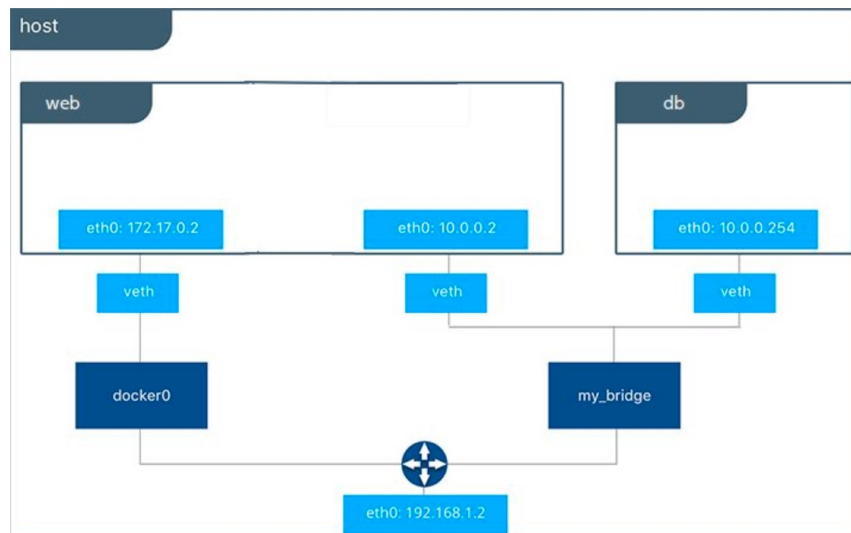


Docker: componenti chiave

- ❑ **Image**: un "template", *read-only*, che contiene tutte le istruzioni necessarie a creare un container docker:
 - può essere definita tramite una lista di istruzioni specificate in un file di testo (**Dockerfile**), oppure ottenuta da un **registry**.
- ❑ **Container**: **istanza eseguibile** di una image:
 - può essere creato, avviato, stoppato, mitigato, etc. ;
 - le modifiche *non* si riflettono sulla image;
 - è possibile definire "quanto" il container è isolato dall'host.
- ❑ **Network** e **volume** (non trattati in questa presentazione).

Docker: networking

- L'utente può creare e gestire reti.



Gestione immagini (alcuni comandi)

- Ricerca in un registro (per es., quello pubblico di default):

```
docker search nginx # nginx è l'immagine cercata
```

- Elenco immagini:

```
docker image ls
```

- Import immagine da un registro:

```
docker image pull base/archlinux # base/archlinux è una immagine
```

- Rimozione immagine locale; rimozione immagini non usate:

```
docker image rm ubuntu # ubuntu è una immagine  
docker image prune
```

Creazione e avvio di un container*

- Pull image e avvio:

```
docker run -it --name ubu1 ubuntu /bin/bash
```

Diagramma di annotazione: un'freccia rivolta verso il basso indica che **image** (in rosa) si riferisce a `ubuntu`; un'freccia rivolta verso l'alto indica che **nome container** (in rosa) si riferisce a `ubu1`. Un'freccia rivolta verso sinistra indica che `/bin/bash` è il comando eseguito dal container (opzionale); se presente, sovrascrive quello specificato nella image (CMD).

- Apertura di una shell *nel* container:

```
docker exec -it nome_container /bin/bash
```

Alcune opzioni

```
-i          // comando interattivo
-t          // allocazione di un terminale
--name      // nome del container
-d          // il container esegue in background
-p h:e      // mappa la porta e (esposta) sulla porta h (dell'host)
-P          // pubblica le porte esposte su porte random dell'host
```

*Docker run reference: <https://docs.docker.com/engine/reference/run/>

Gestione container*

- Mostra tutti i container in esecuzione o in qualsiasi stato

```
docker ps
docker ps -a
```

- Stop o remove di un container:

```
docker stop ubu1          # ubu1 è il nome di un container
docker rm ubu1
```

- Mostra i mapping delle porte (**port**):

```
docker port myapplication # myapplication è il nome di un
5000/tcp -> 0.0.0.0:32786  # container
5000/tcp -> [::]:32786
```

*Docker run reference: <https://docs.docker.com/engine/reference/run/>

Esempio 1

- Esecuzione di un web server ([nginx](#)), tramite immagine scaricata dal registry ([image pull](#)):

```
[studente@opsys:~/dockertest]$ docker image pull nginx
```

```
Using default tag: latest
```

```
latest: Pulling from library/nginx
```

```
... // omissso
```

```
[studente@opsys:~/dockertest]$ docker run -P -d --name MyWebServer nginx
```

```
71cc66fe9a8373cc791e321ab624f603fb31bf35db983686e0cf8049b591dbc9
```

```
[studente@opsys:~/dockertest]$ docker port MyWebServer
```

```
80/tcp -> 0.0.0.0:32789
```

```
80/tcp -> [::]:32789
```

porta host ottenuta
con [docker port](#)



Il browser è quello dell'host. L'indirizzo della VM in esempio è 192.168.64.2 (visualizzabile tramite [ifconfig](#) eseguito nella VM).

Attenzione alla modalità di networking usata dalla VM (NAT vs bridge).

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to [nginx.org](#).
Commercial support is available at [nginx.com](#).

Thank you for using nginx.

Esempio 2

- Avvio di un container ed apertura della [shell bash](#) del container.

```
[studente@opsys:~/dockertest]$ docker run -it ubuntu bash
```

```
[root@aa2d385d39c7:/#] ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	1.7	0.0	4624	3516	pts/0	Ss	12:32	0:00	bash
root	9	0.0	0.0	7060	1608	pts/0	R+	12:32	0:00	ps aux

```
[root@aa2d385d39c7:/#] █
```


NOTA: Dockerfile (1/2)

- ❑ File di testo che contiene la "ricetta" per costruire la *image*;
- ❑ Definisce un componente applicativo e contiene solo il software ad esso necessario. **Dockerfile** relativo all'esercizio*:

```
FROM ubuntu:latest          # FROM: specifica la 'base' della image
RUN apt-get update -y       # RUN: usato durante il build di una image
RUN apt-get install -y python3-pip      (per es., per installare pacchetti)
RUN pip install flask --break-system-packages

ADD "application.py" "/tmp/application.py" # ADD: copia dall'host al container
EXPOSE 5000                          # EXPOSE: indica che l'applicazione
                                      # nel container è in ascolto su una porta
CMD [ "python3", "/tmp/application.py" ] # CMD: comando da eseguire nel container
                                      quando esso è avviato.
```

*[application.py](#) disponibile su Handy

NOTA: Dockerfile (2/2)

- ❑ **Build** di una image da Dockerfile. Posizionarsi nella directory contenente il **Dockerfile** ed eseguire:

```
docker build -t myimg .      # myimg specifica il nome (tag) della image
```

- ❑ La nuova immagine verrà salvata con le altre disponibili sull'host;
- ❑ **Dockerfile reference** completa:
<https://docs.docker.com/engine/reference/builder>

Esempio 3

- Build di una image da **Dockerfile** ed esecuzione:

```
[studente@opsys:~/dockertest$ ls
application.py  Dockerfile
[studente@opsys:~/dockertest$ docker build -t mywebapp .
... // omissio
Successfully built 4478c2f1532a
Successfully tagged mywebapp:latest
[studente@opsys:~/dockertest$ docker run -P -d --name ApplicationTest mywebapp
ecd34ded687c0242cb6f67b91245ab763a72e0823c82bcaeb445d9132ac20b9b
[studente@opsys:~/dockertest$ docker port ApplicationTest
5000/tcp -> 0.0.0.0:32790
5000/tcp -> [::]:32790
```

<http://192.168.64.2:32790/welcome?studente=Mario%20Rossi&corso=Sistemi%20Operativi>



Benvenuto

Esempio di applicazione.

Utilizzo: IP_ADDR:PORTA/welcome?studente=Rossi&corso=SO



Saluto personalizzato

Ciao **Mario Rossi**, benvenuto al corso di **Sistemi Operativi**!

Docker ps e rm

- È necessario arrestare e rimuovere un container prima di poterne avviare uno con lo stesso nome.

```
studente@opsys:~/dockertest$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
ad6c3461f364   mywebapp  "python3 /tmp/applic..." 6 seconds ago  Up 2 seconds  0.0.0.0:32769->5000/tcp, :::32769->5000/tcp  ApplicationTest
7f5b6c7e4ab5   nginx     "/docker-entrypoint..." 11 seconds ago Up 6 seconds  0.0.0.0:32768->80/tcp, :::32768->80/tcp    MyWebServer
studente@opsys:~/dockertest$ docker stop ApplicationTest
ApplicationTest
studente@opsys:~/dockertest$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
ad6c3461f364   mywebapp  "python3 /tmp/applic..." About a minute ago  Exited (137) 37 seconds ago  0.0.0.0:32768->80/tcp, :::32768->80/tcp  ApplicationTest
7f5b6c7e4ab5   nginx     "/docker-entrypoint..." About a minute ago  Up About a minute  0.0.0.0:32768->80/tcp, :::32768->80/tcp    MyWebServer
studente@opsys:~/dockertest$ docker rm ApplicationTest
ApplicationTest
studente@opsys:~/dockertest$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
7f5b6c7e4ab5   nginx     "/docker-entrypoint..." 2 minutes ago  Up About a minute  0.0.0.0:32768->80/tcp, :::32768->80/tcp  MyWebServer
studente@opsys:~/dockertest$ docker run -P -d --name MyWebServer nginx
docker: Error response from daemon: Conflict. The container name "/MyWebServer" is already in use by container "7f5b6c7e4ab5e25d8ee0185f6518015bca177b8c8745e6fca813c6108397220b". You have to remove (or rename) that container to be able to reuse that name.
See 'docker run --help'.
```

// arresto del container

// rimozione del container

// tentativo (errato) di avviare il container MyWebServer (nome già in uso).